

Pertanyaan Sidang — Sistem Informasi Magang Terpadu (SIDUL)

> Dokumentasi ini berisi daftar pertanyaan kritis yang mungkin diajukan oleh dosen penguji pada sidang skripsi. Pertanyaan dikelompokkan berdasarkan tiga ranah utama: **Basis Data**, **Backend**, dan **Frontend**. Setiap pertanyaan dilengkapi dengan jawaban yang benar dan relevan terhadap implementasi aktual SIDUL.

A. Basis Data (Database)

A.1 Migrasi dan Perancangan

Q1: Anda menggunakan migration `2026_06_04_104554_add_draft_status_to_laporans_table.php` yang melakukan `ALTER TABLE MODIFY COLUMN`. Mengapa migration ini menyebabkan error saat diuji dengan SQLite, dan bagaimana cara Anda memperbaikinya?

Jawaban:

SQLite tidak mendukung perintah `ALTER TABLE MODIFY COLUMN`. SQLite hanya mendukung `ALTER TABLE RENAME TO` dan `ALTER TABLE ADD COLUMN`. Perbaikannya adalah dengan mengecek driver database menggunakan `Schema::getConnection()->getDriverName()`; jika driver adalah `sqlite`, gunakan sintaks `ALTER TABLE ... ADD COLUMN ...` saja; jika `mysql`, gunakan `MODIFY COLUMN`. Atau pendekatan yang lebih aman: buat tabel baru, salin data, hapus tabel lama, rename tabel baru.

Q2: Anda memiliki relasi `mahasiswas.dosen_wali_id` yang mengacu ke `dosens.id`. Namun di sisi Dosen tidak ada relasi `hasMany` ke Mahasiswa. Mengapa? Apakah ini tidak mengganggu query untuk menampilkan mahasiswa bimbingan?

Jawaban:

Relasi `hasMany` dari Dosen ke Mahasiswa memang tidak didefinisikan secara eksplisit di model Dosen. Query mahasiswa bimbingan di `DosenService::getMhsBimbingan()` dilakukan dengan pendekatan berbeda: mengambil semua mahasiswa yang memiliki `dosen_wali_id` sama dengan ID dosen yang sedang login. Ini bisa dilakukan tanpa relasi Eloquent `hasMany`, cukup dengan `Mahasiswa::where('dosen_wali_id', $dosenId)`. Meskipun demikian, menambahkan relasi `hasMany` akan membuat kode lebih ekspresif dan memanfaatkan fitur

Eloquent secara maksimal.

Q3: Mengapa Anda memilih tipe data `enum` untuk kolom `status_magang` di tabel `mahasiswas` dan `magangs`? Apa kelemahan penggunaan `enum` di database?

Jawaban:

`Enum` dipilih untuk memastikan integritas data — hanya nilai tertentu yang valid ('Pending', 'Approve', 'Rejected' untuk mahasiswa; 'Pending', 'Aktif', 'Selesai', 'Ditolak' untuk magang). Namun kelemahannya: (1) menambah nilai baru membutuhkan migration `ALTER TABLE ... MODIFY COLUMN` yang tidak kompatibel dengan SQLite; (2) portabilitas database rendah; (3) jika ada logika bisnis yang bertambah, pengelolaan nilai enum menyebar antara database dan kode. Alternatif yang lebih baik adalah menggunakan string biasa dengan validasi di level Laravel (`Rule::in(...)`), atau membuat tabel master status terpisah.

Q4: Di tabel `settings`, Anda menyimpan key-value seperti `is_periode_open`. Apa kelebihan dan kekurangan pendekatan ini dibandingkan menyimpannya di file `.env` atau di tabel terpisah dengan kolom boolean?

Jawaban:

Kelebihan: (1) nilai dapat diubah oleh operator melalui antarmuka tanpa perlu deploy ulang atau akses server; (2) riwayat perubahan tersimpan di database. Kekurangan: (1) setiap kali halaman di-load, sistem harus query ke database (bisa di-cache dengan `Cache::remember()`); (2) tidak ada type-casting otomatis (nilai disimpan sebagai string '0'/'1'). `.env` lebih cocok untuk konfigurasi lingkungan (APP_ENV, DB_HOST), bukan untuk fitur yang diubah oleh pengguna secara dinamis.

Q5: Tabel `edit_requests` memiliki kolom `old_value` dan `new_value`. Namun di kode, Anda menyimpan data dalam format pipe-separated (`mulai|selesai`) untuk field `periode_magang`. Mengapa tidak menggunakan JSON saja? Apa implikasinya terhadap query pencarian?

Jawaban:

Format pipe-separated lebih sederhana untuk kasus dua nilai (tanggal mulai dan selesai), dan lebih mudah dibaca manusia di database. Namun kelemahannya: (1) tidak standar — setiap fitur yang berbeda bisa menggunakan separator berbeda; (2) query `LIKE` untuk mencari nilai tertentu tidak efisien dan rawan false positive; (3) jika jumlah nilai bertambah di masa depan, format ini sulit dikembangkan. JSON akan lebih terstruktur, mendukung validasi otomatis, dan lebih mudah diparsing di kode. Namun JSON juga tidak bisa di-query per-key secara native di

MySQL versi lama.

A.2 Optimasi dan Query

Q6: Di ``MahasiswaService::getCurrentMahasiswa()``, Anda melakukan query ``User::with('mahasiswa')->where('id', $userId)->first()``. Apakah ini sudah efisien? Apa yang terjadi jika tabel `users` memiliki jutaan baris?

Jawaban:

Query ini cukup efisien untuk skala kecil karena menggunakan primary key (``id``). Namun ``with('mahasiswa')`` melakukan eager loading yang menghasilkan 2 query (`users JOIN mahasiswas`). Untuk skala besar, ini masih OK selama kolom ``id`` di-index (primary key). Alternatif: ``Mahasiswa::where('user_id', $userId)->first()`` langsung tanpa melalui `User` — hanya 1 query. Pendekatan yang lebih baik juga bisa menggunakan ``firstOrFail`` untuk langsung 404 jika tidak ditemukan, menghindari pengecekan ``if (!$mahasiswa)`` manual.

Q7: Di fitur `monitoring operator`, Anda menghitung progress di Blade template secara inline. Apa kelemahan pendekatan ini dan bagaimana seharusnya?

Jawaban:

Kelemahan: (1) logika bisnis (rules perhitungan progress) tercampur dengan lapisan presentasi — melanggar prinsip Separation of Concerns; (2) jika aturan progress berubah, harus mencari di semua Blade file, bukan di satu service; (3) tidak bisa di-reuse oleh controller lain atau API. Seharusnya logika ini dipindahkan ke method di Model ``Magang`` (misal ``getProgressAttribute()``) atau ke Service class, sehingga Blade hanya memanggil ``$magang->progress``.

A.3 Integritas Data

Q8: Ketika dosen menyetujui laporan (``dosen/laporan/{id}/approve``), status magang berubah menjadi `'Selesai'`. Apakah ada mekanisme yang mencegah perubahan data setelah magang Selesai? Bagaimana jika operator tidak sengaja menghapus magang yang sudah Selesai?

Jawaban:

Di kode saat ini, penghapusan magang oleh operator menggunakan ``Magang::destroy($id)`` tanpa pengecekan status. Jika magang dengan status `'Selesai'` dihapus, data laporan, logbook, dan peserta magang akan ikut terhapus (jika ada foreign key cascade) atau menjadi orphan.

Seharusnya ada validasi: hanya magang dengan status 'Pending' yang boleh dihapus. Tidak ada proteksi di level database (seperti RESTRICT) untuk mencegah hal ini.

Q9: Apakah ada jaminan bahwa satu mahasiswa hanya terdaftar di satu magang aktif? Bagaimana Anda menanganinya di kode dan di database?

Jawaban:

Di level database, tidak ada unique constraint yang mencegah satu `mahasiswa\_id` muncul di beberapa `peserta\_magangs`. Proteksi hanya dilakukan di level aplikasi (service layer) dengan mengecek apakah mahasiswa sudah memiliki `pesertaMagang` terkait. Ini berarti dua request yang hampir bersamaan bisa menyebabkan race condition. Solusi yang lebih aman: tambahkan `unique constraint` di tabel `peserta\_magangs` pada kolom `mahasiswa\_id`, atau gunakan database transaction dengan `lockForUpdate()`.

B. Backend (Laravel)

B.1 Arsitektur dan Pola Desain

Q10: Anda menggunakan Service layer (`MahasiswaService`, `DosenService`, `OperatorService`). Apa alasan Anda memisahkan logika bisnis dari Controller? Apakah Anda juga menerapkan Repository pattern?

Jawaban:

Pemisahan Service dari Controller bertujuan agar Controller tetap "tipis" — hanya bertugas menerima request, memanggil service, dan mengembalikan response. Service berisi logika bisnis yang bisa di-reuse oleh beberapa controller atau command. Di SIDUL, Repository pattern tidak digunakan — akses database tetap dilakukan langsung di Service menggunakan Eloquent Model. Ini adalah keputusan arsitektur yang umum untuk aplikasi skala kecil-sedang. Untuk proyek yang lebih besar, Repository bisa ditambahkan untuk abstraksi database.

Q11: Controller `MahasiswaController` memiliki 12 method (index: dashboard, pendaftaran, logbook, etc.). Apakah ini tidak terlalu besar? Bagaimana prinsip *Single Responsibility* di sini?

Jawaban:

Idealnya, satu controller bertanggung jawab atas satu resource. `MahasiswaController` menangani dashboard, pendaftaran, logbook, laporan, edit data, dan cetak PDF — terlalu banyak tanggung jawab. Seharusnya dipecah menjadi: - `DashboardController` — dashboard

dan redirect - `PendaftaranController` — pendaftaran magang - `LogbookController` — CRUD logbook - `LaporanController` — CRUD laporan - `EditDataController` — pengajuan edit data

Namun untuk skala aplikasi ini, satu controller masih bisa diterima karena method-methodnya relatif sederhana.

Q12: Di `AuthController`, Anda menangani login untuk 4 role berbeda dalam satu method. Bagaimana cara sistem menentukan ke mana user akan di-redirect setelah login?

Jawaban:

Setelah kredensial diverifikasi, sistem mengecek properti `role` dari user yang login. Jika role adalah 'admin', redirect ke `/dashboard/admin`; jika 'dosen' ? `/dashboard/dosen`; 'operator' ? `/dashboard/operator`; 'mahasiswa' ? `/mahasiswa/dashboard`. Ini diimplementasikan dengan statement `if-else` atau `match()` di method `login` atau di `authenticated()` method bawaan Laravel.

B.2 Middleware dan Keamanan

Q13: Bagaimana Anda mengimplementasikan Role-Based Access Control (RBAC)? Apakah ada middleware khusus untuk setiap role, atau satu middleware dengan parameter?

Jawaban:

Bisa dicek di rute. Biasanya ada middleware seperti `role:admin` atau `role:mahasiswa` yang menerima parameter role. Di Laravel, ini dibuat dengan `php artisan make:middleware RoleMiddleware` dan didaftarkan di `Kernel.php`. Route-nya akan seperti:

```
Route::middleware(['auth', 'role:admin'])->group(...)`
```

Middleware ini membandingkan `\$request->user()->role` dengan role yang diizinkan. Jika tidak cocok, redirect ke dashboard role yang bersangkutan.

Q14: Apakah Anda menggunakan mekanisme otorisasi seperti `Gate` atau `Policy`? Atau hanya mengandalkan middleware role?

Jawaban:

Hanya menggunakan middleware role. `Gate` dan `Policy` tidak diimplementasikan. Gate/Policy berguna untuk otorisasi yang lebih granular — misalnya "dosen hanya bisa approve laporan mahasiswa bimbingannya sendiri", bukan "semua dosen bisa approve semua laporan". Di SIDUL, logika "hanya bimbingan sendiri" sudah ditangani di Service layer dengan query `where('dosen\_pembimbing\_id', \$dosenId)`, jadi Gate/Policy tidak diperlukan secara ketat, tetapi akan membuat kode lebih eksplisit dan testable.

Q15: Apakah ada proteksi terhadap CSRF, XSS, dan SQL Injection di aplikasi Anda? Jelaskan implementasinya.

Jawaban:

- **CSRF:** Semua form di Blade menggunakan `@csrf` yang otomatis menyertakan token CSRF. Laravel menolak request tanpa token yang valid. Middleware `VerifyCsrfToken` sudah aktif secara default. - **XSS:** Blade secara otomatis men-escape output dengan sintaks `{{ \$var }}`. Untuk konten CKEditor yang mengandung HTML, SIDUL menggunakan `{!! \$var !!}` (unescaped) — ini memang berpotensi XSS jika tidak hati-hati. Namun CKEditor memiliki sanitasi di sisi klien. - **SQL Injection:** Terproteksi karena Laravel menggunakan Eloquent ORM dengan parameter binding. Query seperti `where('nim', \$nim)` akan di-escape otomatis. Tidak ada raw SQL tanpa parameter binding di kode SIDUL.

B.3 Validasi dan Error Handling

Q16: Di method `storePendaftaran`, validasi dilakukan di Controller. Apakah seharusnya validasi dipindahkan ke Service? Atau dibuat Form Request khusus? Apa kelebihan masing-masing?

Jawaban:

- **Validasi di Controller:** Sederhana, cocok untuk validasi sederhana. Namun mencampur logika validasi dengan logika bisnis. - **Form Request:** Dipisah ke kelas sendiri, reusable untuk validasi yang sama di method berbeda, memiliki method `authorize()` untuk otorisasi, dan pesan error bisa dikustom per field. Ini yang paling direkomendasikan untuk proyek dengan validasi kompleks seperti ini. - **Validasi di Service:** Berguna jika validasi memerlukan akses ke database atau logika bisnis lain. Namun kurang tepat karena Service seharusnya menerima data yang sudah valid.

Rekomendasi untuk SIDUL: buat `StorePendaftaranRequest` yang menangani validasi, sehingga Controller tetap tipis.

Q17: Ketika validasi gagal, Anda mengembalikan `back()->with('error', $message)`. Namun untuk validasi form, Anda juga menggunakan `$response->assertSessionHasErrors()`. Apakah konsisten? Bagaimana dengan error handling untuk request AJAX?

Jawaban:

Untuk form biasa, Laravel otomatis me-redirect back dengan error session jika validasi gagal — tidak perlu manual. Untuk pesan error bisnis (bukan validasi form), menggunakan `with('error', ...)` sudah tepat. SIDUL tidak memiliki endpoint AJAX, sehingga error handling JSON belum diimplementasikan. Jika ada endpoint AJAX di masa depan, perlu mengembalikan response JSON dengan status code 422 untuk error validasi.

B.4 Testing

Q18: Anda menggunakan SQLite in-memory untuk pengujian, tetapi aplikasi berjalan di MySQL. Apa risiko dari perbedaan database ini? Pernah menemukan bug yang hanya muncul di MySQL tetapi tidak terdeteksi di SQLite?

Jawaban:

Risiko utama: (1) perbedaan sintaks SQL — migration `MODIFY COLUMN` gagal di SQLite (sudah diperbaiki); (2) perbedaan fitur — MySQL mendukung `FULLTEXT INDEX`, SQLite tidak; (3) perbedaan collation/encoding; (4) transaksi dan locking behavior berbeda. Bug yang terdeteksi: migration `add_draft_status_to_laporans_table` gagal di SQLite karena menggunakan `MODIFY COLUMN`. Solusi: gunakan database yang sama (dengan Docker atau Github Actions service MySQL) untuk CI, atau minimal jalankan satu kali test suite penuh di MySQL sebelum rilis.

Q19: Dari 86 test yang ada, berapa persen kode Anda yang tercakup (*code coverage*)? Apakah Anda menggunakan tools seperti Xdebug atau PHPUnit coverage?

Jawaban:

Code coverage belum diukur. PHPUnit memiliki fitur `--coverage-html` yang membutuhkan Xdebug atau PCOV. Tanpa code coverage, tidak diketahui bagian kode mana yang belum diuji — misalnya skenario error, edge cases, atau fitur baru. Idealnya, coverage di atas 70% untuk controller dan service. Method yang belum teruji: `storeEditData` di MahasiswaController, `togglePeriode` error scenarios, dan beberapa kondisi cabang di service layer.

C. Frontend (Blade, Tailwind, DaisyUI)

C.1 Arsitektur Frontend

Q20: Anda menggunakan Vite sebagai build tool. Jelaskan peran Vite dalam pengembangan frontend SIDUL. Apa yang terjadi jika Anda menjalankan ``npm run build``?

Jawaban:

Vite berfungsi sebagai module bundler dan asset compiler. Saat development (``npm run dev``), Vite menyediakan Hot Module Replacement (HMR) sehingga perubahan CSS/JS langsung terlihat tanpa reload halaman. Saat build (``npm run build``), Vite: (1) memproses file CSS dari Tailwind (scanning class yang digunakan, menghasilkan CSS final yang sudah **purged**); (2) menggabungkan dan minify file JS; (3) menambahkan versioning/hashing pada filename untuk cache busting. Hasil build disimpan di ``public/build/``.

Q21: Anda memilih Tailwind CSS + DaisyUI dibandingkan Bootstrap. Apa kelebihan dan kekurangan Tailwind untuk proyek seperti SIDUL?

Jawaban:

Kelebihan: (1) ukuran file CSS final kecil karena utility yang tidak dipakai akan di-purge; (2) desain lebih fleksibel dan tidak terikat komponen Bootstrap; (3) konsistensi ukuran, warna, spacing terjaga lewat konfigurasi ``tailwind.config.js``; (4) DaisyUI menyediakan komponen siap pakai (btn, card, modal, badge, progress). Kekurangan: (1) HTML lebih panjang karena penuh utility class; (2) kurva belajar untuk memahami utility class; (3) untuk developer yang terbiasa Bootstrap, perlu adaptasi.

Q22: Di halaman laporan, Anda menggunakan CKEditor 5 yang di-load dari CDN. Apa risiko menggunakan CDN dibandingkan instalasi lokal via npm?

Jawaban:

Risiko: (1) dependensi pada koneksi internet — jika server tidak punya akses internet, CKEditor tidak akan muncul; (2) versi CDN bisa berubah tanpa pemberitahuan jika tidak di-pin; (3) latency — setiap kali halaman di-load, browser harus mendownload CKEditor; (4) tidak bisa dikustomisasi secara mendalam. Seharusnya CKEditor diinstal via ``npm install @ckeditor/ckeditor5-build-classic`` lalu di-import sebagai modul, sehingga masuk dalam proses Vite build.

C.2 Komponen dan Reusability

Q23: Anda memiliki beberapa komponen Blade seperti ``. Apakah komponen-komponen ini sudah reusable dan konsisten di semua halaman? Beri contoh komponen yang masih duplikasi kode.

Jawaban:

Sebagian besar komponen sudah reusable (`x-card`, `x-button`, `x-modal`). Namun ada duplikasi yang terlihat: (1) banner error/success di halaman pendaftaran dan laporan menggunakan struktur HTML yang sama (`bg-red-50 border-2 border-red-200 p-5 rounded-2xl ...`) tetapi ditulis ulang di setiap halaman — seharusnya dibuat komponen `` atau ``; (2) tabel monitoring di operator dan dosen memiliki struktur yang mirip tetapi ditulis terpisah.

Q24: Di halaman pendaftaran, terdapat logika validasi NIM anggota kelompok di JavaScript (client-side). Apakah validasi ini juga dilakukan di server-side? Bagaimana jika pengguna menonaktifkan JavaScript?

Jawaban:

Validasi client-side (JS) hanya untuk kenyamanan pengguna — memberikan feedback cepat tanpa reload halaman. Validasi server-side tetap dilakukan di Controller untuk keamanan, karena pengguna bisa menonaktifkan JS atau mengirim request langsung via Postman/cURL. Di SIDUL, validasi server-side mencakup: NIM harus terdaftar di tabel `mahasiswas`, status anggota harus 'Approve', anggota belum terdaftar di magang lain, dan jumlah anggota tidak melebihi batas.

C.3 Performa dan UX

Q25: Di halaman monitoring operator, Anda mengimplementasikan fitur **real-time search menggunakan JavaScript di sisi klien — semua data sudah di-load dari server, lalu client-side yang menyaring. Apa kelemahan pendekatan ini untuk data dalam jumlah besar (misal 10.000 mahasiswa)?**

Jawaban:

Pendekatan client-side search hanya cocok untuk data dalam jumlah kecil hingga sedang (ratusan baris). Untuk 10.000 mahasiswa, mengirim seluruh data sekaligus akan: (1) memperbesar ukuran halaman (bisa > 5 MB HTML); (2) memperlambat rendering karena DOM terlalu besar; (3) search client-side menjadi lambat karena harus menyaring 10.000 elemen DOM. Solusi yang lebih baik: gunakan server-side search/pagination dengan AJAX atau Livewire, di mana setiap pencarian mengirim request ke server dan hanya mengembalikan

data yang cocok.

Q26: Halaman laporan menggunakan 4 instance CKEditor yang masing-masing cukup berat. Apakah ada masalah performa yang Anda amati? Bagaimana solusinya?

Jawaban:

CKEditor cukup berat karena masing-masing instance memuat toolbar, stylesheet, dan engine sendiri. Dengan 4 instance di satu halaman, waktu load halaman bisa lama terutama di koneksi lambat. Solusi: (1) lazy load CKEditor — hanya menginisialisasi instance saat tab bab diklik, bukan saat halaman pertama kali di-load; (2) gunakan textarea biasa dengan preview, dan CKEditor hanya aktif untuk tab yang aktif; (3) pertimbangkan editor alternatif yang lebih ringan seperti TipTap atau Quill jika fitur yang dibutuhkan sederhana.

Q27: SIDUL menggunakan banyak animasi (transisi, hover effect, active:scale-95). Apakah animasi ini berdampak pada performa di perangkat dengan spesifikasi rendah?

Jawaban:

Animasi berbasis CSS (`transition-all`, `transform`, `opacity`) umumnya di-*accelerate* oleh GPU, sehingga tidak signifikan memengaruhi performa. Namun penggunaan `hover:scale-[1.02]` dan `active:scale-95` pada setiap tombol, serta `animate-bounce` dan `animate-pulse` pada badge status, bisa menyebabkan *layout thrashing* jika terlalu banyak elemen yang beranimasi bersamaan. Di perangkat rendah, sebaiknya kurangi jumlah animasi atau gunakan `prefers-reduced-motion` media query untuk menonaktifkan animasi.

C.4 Keamanan Frontend

Q28: Di halaman laporan, data dari CKEditor dirender menggunakan `{!! $var !!}` (unescaped). Apa risiko keamanannya? Bagaimana cara memitigasinya?

Jawaban:

`{!! !!}` tidak melakukan HTML escaping. Jika konten yang disimpan mengandung script berbahaya (misal `<script>`), script akan dieksekusi di browser. Risiko ini sebagian dimitigasi karena: (1) CKEditor memiliki sanitasi di sisi klien (tidak semua tag HTML diizinkan); (2) input sudah melalui validasi server-side. Namun tetap ada risiko jika konten dikirim langsung via API/Postman. Solusi: gunakan HTML Purifier (package `mews/purifier`) untuk membersihkan konten sebelum disimpan, atau gunakan Blade escaped `{{ }}` dengan library markdown sebagai alternatif.

Q29: Apakah ada proteksi untuk mencegah mahasiswa mengakses laporan mahasiswa lain dengan cara mengubah `magang\_id` di form? Jelaskan.

Jawaban:

Form laporan menggunakan `magang\_id` sebagai hidden input. Di sisi server, method `storeLaporan` di MahasiswaController mengambil data dari request. Namun di service layer (`simpanLaporan`), sistem juga mengambil `getCurrentMahasiswa(\$user)` dan mengecek apakah mahasiswa tersebut memiliki pesertaMagang yang terkait. Meskipun `magang\_id` bisa diubah oleh pengguna, service akan tetap memproses laporan hanya untuk magang yang terdaftar ke mahasiswa yang sedang login. Ini adalah bentuk implicit authorization.

D. Pertanyaan Umum & Konseptual

D.1 Pemilihan Teknologi

Q30: Mengapa Anda memilih Laravel dibandingkan framework PHP lain seperti CodeIgniter atau framework JavaScript seperti Next.js?

Jawaban:

Laravel dipilih karena: (1) fitur bawaan lengkap (Eloquent ORM, Blade templating, middleware, auth scaffolding, testing); (2) ekosistem besar (Composer packages, dokumentasi, komunitas); (3) mendukung MVC yang memisahkan logika bisnis dari presentasi; (4) migration system yang memudahkan versioning database; (5) familiarity dengan tim pengembang. CodeIgniter lebih ringan tetapi kurang fitur. Next.js (JavaScript) akan membutuhkan full-stack JavaScript dan API backend terpisah, yang justru menambah kompleksitas untuk proyek skripsi.

Q31: Apakah Anda mempertimbangkan untuk membuat aplikasi ini menjadi SPA (Single Page Application) menggunakan React atau Vue? Mengapa memilih tetap menggunakan Blade?

Jawaban:

Blade dipilih karena: (1) integrasi langsung dengan Laravel — tidak perlu membuat REST API terpisah; (2) server-side rendering — SEO friendly, dan halaman pertama kali dimuat lebih cepat; (3) development lebih cepat karena backend dan frontend dalam satu kodebase; (4) untuk skala aplikasi seperti SIDUL (form-based, CRUD-heavy), SPA tidak memberikan keuntungan signifikan dibanding Blade. SPA akan relevan jika aplikasi membutuhkan interaksi real-time yang kompleks atau harus bekerja offline (PWA).

D.2 Skalabilitas dan Masa Depan

Q32: Jika kampus Anda memiliki 5.000 mahasiswa magang per tahun, apakah arsitektur SIDUL saat ini mampu menangani beban tersebut? Sebutkan bottleneck-nya.

Jawaban:

Beberapa bottleneck: (1) query monitoring operator yang melakukan ``Magang::with(['peserta.mahasiswa', 'pembimbing', 'laporan', 'logbooks'])->get()`` akan berat untuk ribuan record — perlu paging dan indexing; (2) cetak PDF menggunakan DomPDF yang synchronous — jika 100 mahasiswa mencetak bersamaan, server bisa kehabisan memory; (3) client-side search di monitoring dosen mengirim semua data ke browser; (4) tidak ada queue system untuk task berat seperti export CSV atau cetak PDF. Perbaikan: tambahkan pagination di semua halaman, queue untuk task berat, dan index di kolom yang sering di-query.

Q33: Fitur apa yang paling ingin Anda tambahkan jika proyek ini dilanjutkan setelah skripsi?

Jawaban:

(1) Notifikasi real-time menggunakan Laravel Reverb atau Firebase untuk memberitahu mahasiswa saat laporan direview/disetujui; (2) Fitur upload file pendukung (surat lamaran, proposal) yang saat ini tidak ada; (3) Dashboard analitik dengan grafik interaktif (Chart.js atau ApexCharts) untuk pihak prodi; (4) Sistem reminder otomatis untuk mahasiswa yang belum mengisi logbook; (5) API endpoint untuk integrasi dengan sistem akademik kampus (SIKAD).

D.3 Proses Pengembangan

Q34: Berapa lama waktu pengembangan SIDUL? Bagaimana pembagian tugas frontend dan backend?

Jawaban:

Estimasi 3-4 bulan, dengan tahapan: (1) analisis kebutuhan dan perancangan database (2 minggu); (2) setup Laravel dan migration (1 minggu); (3) pengembangan modul auth dan RBAC (1 minggu); (4) modul mahasiswa — pendaftaran, logbook, laporan (3 minggu); (5) modul dosen dan operator (2 minggu); (6) modul admin (1 minggu); (7) frontend styling dengan Tailwind/DaisyUI (2 minggu); (8) pengujian (2 minggu); (9) revisi dan dokumentasi (2 minggu). Frontend dan backend dikembangkan bersamaan karena Blade terintegrasi langsung dengan Laravel.

Q35: Apakah Anda menggunakan version control (Git)? Bagaimana strategi branching Anda?

Jawaban:

Git digunakan untuk version control. Untuk proyek individu (skripsi), strategi branching sederhana: branch `main` berisi kode stabil, dan branch `develop` untuk pengembangan sehari-hari. Fitur baru dikerjakan di branch `feature/\*` (misal `feature/laporan-review`), lalu di-merge ke `develop`. Commit dilakukan setiap kali menyelesaikan satu fungsi kecil dengan pesan commit yang deskriptif, misal "Add validation for kelompok registration max 3 members".

Q36: Jelaskan bagaimana Anda memastikan kualitas kode SIDUL. Apakah Anda menggunakan code sniffing, formatting tools, atau code review?

Jawaban:

Kualitas kode dijaga melalui: (1) PHPUnit test suite (86 tests) yang dijalankan sebelum setiap commit; (2) mengikuti PSR-12 coding standard secara manual (tidak menggunakan Laravel Pint atau PHP CS Fixer); (3) penamaan method dan variabel yang deskriptif dalam Bahasa Inggris; (4) penggunaan Service layer untuk memisahkan logika bisnis. Yang belum dilakukan: (a) tidak ada code review karena proyek individu; (b) tidak ada static analysis (PHPStan, Larastan); (c) tidak ada pre-commit hook untuk otomatisasi formatting.

E. Pertanyaan tentang Dokumentasi dan Skripsi

Q37: Di skripsi Anda, tabel-tabel perancangan (Tabel 3.29–3.34) hanya berisi rencana pengujian. Mengapa Anda tidak menyertakan kolom hasil aktual dan status di tabel yang sama? Pembaca ingin melihat perbandingan antara yang direncanakan dan hasilnya.

Jawaban:

Rencana pengujian di BAB III memang hanya berisi skenario yang direncanakan. Hasil aktual akan disajikan di BAB IV setelah sistem diimplementasikan dan diuji. Namun untuk memudahkan pembaca, kolom "Hasil Aktual" dan "Status" sebaiknya langsung disertakan di tabel yang sama di BAB III, dengan catatan bahwa kolom tersebut akan diisi setelah implementasi. Atau, sajikan tabel lengkap (rencana + hasil) di BAB IV dengan referensi silang ke BAB III.

Q38: Anda menulis bahwa pengujian manual dilakukan oleh 4 tester dengan 233 uji dan hasil 100%. Apakah realistis semua uji lulus 100% tanpa satu pun kegagalan? Jika iya,

skenario uji apa yang sengaja Anda buat untuk menguji batas sistem (negative testing)?

Jawaban:

Tingkat keberhasilan 100% memang mungkin terjadi jika: (1) seluruh bug telah diperbaiki sebelum pengujian final; (2) skenario pengujian memang dirancang berdasarkan fitur yang sudah berfungsi; (3) penguji mengikuti skenario yang sudah ditentukan. Negative testing tetap dilakukan — contohnya: password salah, username tidak terdaftar, field kosong, format file tidak sesuai, akses URL oleh role yang tidak berhak, dan duplikasi data. Namun perlu diakui bahwa cakupan pengujian masih terbatas pada modul yang sudah selesai, dan mungkin ada bug yang tidak terdeteksi karena keterbatasan jumlah tester dan skenario.

Q39: Basis data Anda menggunakan MySQL. Di skripsi, apakah Anda menyertakan diagram ERD? Jika iya, apakah ERD tersebut sesuai dengan implementasi aktual? Jika ada perbedaan, jelaskan.

Jawaban:

ERD (Entity Relationship Diagram) disertakan di BAB III. ERD mencakup tabel: `users`, `mahasiswas`, `dosens`, `magangs`, `peserta\_magangs`, `logbooks`, `laporans`, `edit\_requests`, `revisi\_laporans`, `settings`. ERD aktual sesuai dengan perancangan, dengan beberapa penyesuaian: (1) kolom `kode\_magang` dihasilkan otomatis saat assign dosen pembimbing, bukan diisi manual; (2) tabel `komentar\_laporans` sempat dibuat lalu dihapus melalui migration karena fungsinya diakomodasi oleh kolom `catatan\_dosen` di tabel `laporans`; (3) tabel `revisi\_laporans` dibuat tetapi belum diimplementasikan penuh di antarmuka.

Q40: Kesimpulan skripsi Anda menyatakan bahwa "SIDUL berhasil dikembangkan dan diuji". Apa indikator keberhasilan yang Anda gunakan? Apakah cukup dengan pengujian saja?

Jawaban:

Indikator keberhasilan meliputi: (1) seluruh fitur yang direncanakan berhasil diimplementasikan sesuai spesifikasi; (2) pengujian PHPUnit menunjukkan 86/86 test lulus; (3) pengujian black box oleh 4 tester menunjukkan 233/233 uji berhasil; (4) sistem dapat digunakan secara offline di jaringan lokal kampus; (5) evaluasi pengguna menunjukkan rata-rata skor 3.16 dari 4.00. Namun indikator tambahan yang perlu dipertimbangkan: (a) survei kepuasan pengguna yang lebih formal dengan sampel lebih besar; (b) pengukuran performa (waktu respons halaman, memory usage); (c) pengujian keamanan yang lebih komprehensif (penetration testing).

— *Dokumen ini disusun berdasarkan implementasi aktual sistem SIDUL* —